

● WINDOW FUNCTIONS — The Skill Every ₹10 LPA Interview Tests

What Window Functions Do

Regular Aggregation

GROUP BY collapses rows into one row per group

You lose row-level detail

```
SELECT region, SUM(sales) FROM
orders GROUP BY region
```

Window Function

Window function adds the result as a new column without collapsing rows

You keep every row and add the calculation alongside it

```
SELECT region, sales, SUM(sales) OVER
(PARTITION BY region)
```

Window Function Syntax

```
function_name() OVER (
    PARTITION BY column -- optional: divide into groups
    ORDER BY column     -- optional: define row sequence
    ROWS/RANGE BETWEEN ... -- optional: define frame
)
```

PARTITION BY vs GROUP BY

	GROUP BY	PARTITION BY
What it does	Collapses rows into one per group	Keeps all rows, adds grouped calculation as new column
Row count after	Equal to number of groups	Same as original table
Use case	Summary reports	Row-level analysis with group context

● RANKING FUNCTIONS

ROW_NUMBER, RANK, DENSE_RANK

```

)
SELECT region, product, total_sales
FROM ranked
WHERE rnk <= 3
ORDER BY region, rnk

```

This exact pattern appears in 80% of SQL technical rounds. The interviewer gives you a table and asks for top N per category. This is the answer.

NTILE — Divide Data Into Equal Buckets

```

-- Divide customers into 4 equal segments by purchase value
SELECT
    customer_id,
    total_purchase,
    NTILE(4) OVER (ORDER BY total_purchase DESC) AS quartile
FROM customers

```

Quartile	Label	Use
1	Top 25%	Premium customers
2	Next 25%	High value
3	Next 25%	Medium value
4	Bottom 25%	Low value

LAG AND LEAD — Compare Rows to Previous or Next Row

Syntax

```

LAG(column, offset, default) OVER (PARTITION BY ... ORDER
BY ...)

```

Function	Handles Ties	Example Output	Real Work Situation
ROW_NUMBER	Assigns unique number even to ties	1, 2, 3, 4	Deduplication — keep only first occurrence
RANK	Skips numbers after ties	1, 1, 3, 4	Leaderboard where tied positions skip a rank
DENSE_RANK	No gaps after ties	1, 1, 2, 3	Tier classification where gaps make no sense

Ranking Syntax and Real Examples

```
-- Rank products by sales within each region
SELECT
    region,
    product,
    sales,
    ROW_NUMBER() OVER (PARTITION BY region ORDER BY sales D
ESC) AS row_num,
    RANK() OVER (PARTITION BY region ORDER BY sales D
ESC) AS rank_with_gaps,
    DENSE_RANK() OVER (PARTITION BY region ORDER BY sales D
ESC) AS rank_no_gaps
FROM orders
```

Most Common Interview Pattern — Top N Per Group

```
-- Top 3 products by sales in each region
WITH ranked AS (
    SELECT
        region,
        product,
        SUM(sales) AS total_sales,
        DENSE_RANK() OVER (PARTITION BY region ORDER BY SUM
(sales) DESC) AS rnk
    FROM orders
    GROUP BY region, product
```

RUNNING TOTALS AND MOVING AVERAGES

Running Total

```
SELECT
    order_date,
    sales,
    SUM(sales) OVER (ORDER BY order_date) AS cumulative_sales
FROM orders
ORDER BY order_date
```

Running Total Within a Partition

```
SELECT
    region,
    order_date,
    sales,
    SUM(sales) OVER (PARTITION BY region ORDER BY order_date) AS regional_cumulative
FROM orders
```

3-Month Moving Average

```
SELECT
    month,
    revenue,
    ROUND(
        AVG(revenue) OVER (
            ORDER BY month
            ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
        )
    )
```

```
, 2) AS moving_avg_3m  
FROM monthly_revenue
```

Frame Clause

What It Means

ROWS BETWEEN 2 PRECEDING AND CURRENT ROW

Current row plus 2 rows before it

ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW

All rows from start up to current — running total

ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING

One row before and one after — centered average

FIRST VALUE AND LAST VALUE

-- Show each order alongside the first and latest order in that region

```
SELECT  
    region,  
    order_date,  
    sales,  
    FIRST_VALUE(sales) OVER (PARTITION BY region ORDER BY order_date) AS first_order_value,  
    LAST_VALUE(sales) OVER (  
        PARTITION BY region  
        ORDER BY order_date  
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING  
    ) AS last_order_value  
FROM orders
```

LAST_VALUE requires the frame clause **ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING**. Without it, **LAST_VALUE** returns the current row, not the actual last row. This is a common interview trap.

ADVANCED SUBQUERY PATTERNS

Correlated Subquery

```
-- Find employees earning above their department average
SELECT
    name,
    department,
    salary
FROM employees e1
WHERE salary > (
    SELECT AVG(salary)
    FROM employees e2
    WHERE e2.department = e1.department
)
```

A correlated subquery runs once for every row in the outer query. It references the outer query using an alias. Interviewers use this to test whether you understand query execution.

EXISTS vs IN — The Interview Distinction

	IN	EXISTS
How it works	Checks if value is in a list	Checks if any row satisfies a condition
Performance on large data	Slower — evaluates entire subquery first	Faster — stops as soon as one match found
Handles NULL	Unreliable with NULLs in subquery	Handles NULLs correctly
When to use	Small subquery result sets	Large tables, checking existence

```
-- EXISTS version -- faster on large tables
SELECT name
FROM customers c
WHERE EXISTS (
    SELECT 1 FROM orders o
    WHERE o.customer_id = c.id
)
```

```
        AND o.order_date >= '2024-01-01'
    )
```

Lateral Join — For Advanced Use Cases

```
-- Get the 2 most recent orders for each customer
SELECT
    c.name,
    o.order_date,
    o.sales
FROM customers c
CROSS JOIN LATERAL (
    SELECT order_date, sales
    FROM orders
    WHERE customer_id = c.id
    ORDER BY order_date DESC
    LIMIT 2
) o
```

PIVOTING DATA IN SQL

Dynamic Pivot Using CASE WHEN

```
-- Convert rows to columns: monthly sales in one row per re
gion
SELECT
    region,
    SUM(CASE WHEN month = 'Jan' THEN sales ELSE 0 END) AS J
an,
    SUM(CASE WHEN month = 'Feb' THEN sales ELSE 0 END) AS F
eb,
    SUM(CASE WHEN month = 'Mar' THEN sales ELSE 0 END) AS M
ar,
    SUM(CASE WHEN month = 'Apr' THEN sales ELSE 0 END) AS A
pr
```

```
FROM orders
GROUP BY region
```

This is the most common data reshaping question in SQL interviews. The interviewer gives you long-format data and asks you to produce a wide-format summary. CASE WHEN inside SUM is the answer every time.

DEDUPLICATION PATTERNS

Keep Only the Most Recent Record Per ID

```
WITH ranked AS (
    SELECT *,
        ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY
Y updated_at DESC) AS rn
    FROM customers
)
SELECT *
FROM ranked
WHERE rn = 1
```

Find and Count Duplicates

```
SELECT
    email,
    COUNT(*) AS occurrences
FROM customers
GROUP BY email
HAVING COUNT(*) > 1
ORDER BY occurrences DESC
```

Delete Duplicates Keep One

```
DELETE FROM customers
WHERE id NOT IN (
    SELECT MIN(id)
    FROM customers
    GROUP BY email
)
```

GAP AND ISLAND PROBLEMS

Find Consecutive Active Days — Classic Interview Problem

```
WITH numbered AS (
    SELECT
        user_id,
        login_date,
        ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date) AS rn
    FROM logins
),
grouped AS (
    SELECT
        user_id,
        login_date,
        DATE_SUB(login_date, INTERVAL rn DAY) AS grp
    FROM numbered
)
SELECT
    user_id,
    MIN(login_date) AS streak_start,
    MAX(login_date) AS streak_end,
    COUNT(*) AS consecutive_days
FROM grouped
GROUP BY user_id, grp
```

```
HAVING COUNT(*) >= 3
ORDER BY user_id, streak_start
```

Gap and island problems appear in product analytics and growth team interviews. The pattern: subtract row number from date. Consecutive dates produce the same group value.

● THE 10 ADVANCED SQL PATTERNS EVERY ₹10 LPA INTERVIEW TESTS

#	Pattern	Interview Question It Answers
1	CTE with multiple steps	Any multi-step analysis question
2	DENSE_RANK with PARTITION BY	Top N per category
3	LAG for month over month	Revenue growth calculation
4	ROW_NUMBER for deduplication	Keep latest record per ID
5	SUM OVER with running total	Cumulative revenue over time
6	CASE WHEN inside SUM	Pivot rows to columns
7	EXISTS vs IN distinction	Performance and NULL handling
8	Correlated subquery	Above average per group
9	Moving average with frame clause	Smoothed trend analysis
10	Gap and island with date subtraction	Consecutive activity streaks

● INTERVIEW QUESTIONS — Full Working Answers

Q1 — Second highest salary without using LIMIT

```
SELECT MAX(salary)
FROM employees
```

```
LEAD(column, offset, default) OVER (PARTITION BY ... ORDER BY ...)
```

Month Over Month Revenue Comparison

```
WITH monthly AS (  
    SELECT  
        DATE_FORMAT(order_date, '%Y-%m') AS month,  
        SUM(sales) AS revenue  
    FROM orders  
    GROUP BY DATE_FORMAT(order_date, '%Y-%m')  
)  
SELECT  
    month,  
    revenue,  
    LAG(revenue) OVER (ORDER BY month)  
AS prev_month_revenue,  
    revenue - LAG(revenue) OVER (ORDER BY month)  
AS absolute_change,  
    ROUND(  
        (revenue - LAG(revenue) OVER (ORDER BY month))  
        / LAG(revenue) OVER (ORDER BY month) * 100  
    , 2)  
AS mom_growth_pct  
FROM monthly  
ORDER BY month
```

LAG and LEAD Real Work Situations

Function	Real Work Situation
LAG(sales, 1)	Compare this month revenue to last month
LAG(sales, 12)	Compare this month to same month last year
LEAD(sales, 1)	Show what next month revenue was alongside current
LAG with PARTITION BY	Compare within each region separately

```
        / LAG(revenue) OVER (ORDER BY month) * 100
    , 2) AS growth_pct
FROM monthly
```

Q5 — Users with 3 or more consecutive login days

```
WITH numbered AS (
    SELECT user_id, login_date,
           ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date) AS rn
    FROM logins
),
grouped AS (
    SELECT user_id, login_date,
           DATE_SUB(login_date, INTERVAL rn DAY) AS grp
    FROM numbered
)
SELECT DISTINCT user_id
FROM grouped
GROUP BY user_id, grp
HAVING COUNT(*) >= 3
```

Built by Abhishek — Every query here comes from real interview rounds and real analyst work. Not textbook. Not YouTube. Actual business problems.

Concept	Explanation
When to use it	Any time your query needs more than two logical steps
Interview signal	Using CTEs instead of nested subqueries signals seniority immediately

CTE Syntax

```
WITH cte_name AS (  
    SELECT  
        column1,  
        column2  
    FROM table_name  
    WHERE condition  
)  
SELECT *  
FROM cte_name  
WHERE another_condition
```

Multiple CTEs in One Query

```
WITH  
    regional_sales AS (  
        SELECT region, SUM(sales) AS total_sales  
        FROM orders  
        GROUP BY region  
    ),  
    regional_orders AS (  
        SELECT region, COUNT(*) AS total_orders  
        FROM orders  
        GROUP BY region  
    )  
SELECT  
    s.region,  
    s.total_sales,  
    o.total_orders,  
    ROUND(s.total_sales / o.total_orders, 2) AS avg_order_v
```

```
WHERE salary < (SELECT MAX(salary) FROM employees)
```

Q2 — Nth highest salary for any N

```
WITH ranked AS (  
    SELECT salary,  
           DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk  
    FROM employees  
)  
SELECT salary  
FROM ranked  
WHERE rnk = 3 -- change 3 to any N
```

Q3 — Customers who bought in both January and February

```
SELECT customer_id  
FROM orders  
WHERE MONTH(order_date) IN (1, 2)  
GROUP BY customer_id  
HAVING COUNT(DISTINCT MONTH(order_date)) = 2
```

Q4 — Month over month growth percentage

```
WITH monthly AS (  
    SELECT  
        DATE_FORMAT(order_date, '%Y-%m') AS month,  
        SUM(sales) AS revenue  
    FROM orders  
    GROUP BY DATE_FORMAT(order_date, '%Y-%m')  
)  
SELECT  
    month,  
    revenue,  
    ROUND(  
        (revenue - LAG(revenue) OVER (ORDER BY month))
```

```

    alue
FROM regional_sales s
JOIN regional_orders o ON s.region = o.region
ORDER BY s.total_sales DESC

```

This pattern — two CTEs joined together — appears in almost every senior SQL interview. Practice writing it from memory.

Recursive CTE — For Hierarchical Data

```

-- Find all employees and their reporting chain
WITH RECURSIVE employee_hierarchy AS (
    -- Anchor: start with top level manager
    SELECT id, name, manager_id, 1 AS level
    FROM employees
    WHERE manager_id IS NULL

    UNION ALL

    -- Recursive: add each level below
    SELECT e.id, e.name, e.manager_id, eh.level + 1
    FROM employees e
    JOIN employee_hierarchy eh ON e.manager_id = eh.id
)
SELECT * FROM employee_hierarchy
ORDER BY level, name

```

Real Work Situation

Org chart — employee reports to manager who reports to director

Product category hierarchy

Bill of materials

Why Recursive CTE

Data is self-referencing, cannot be done with regular JOIN

Parent category contains sub-categories

Component made of sub-components